

c SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. **(5 marks)**

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

Code of Conduct:

I T.Chaitra(192425217) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T.Chaitra

RUBRICS FOR CALCULATION

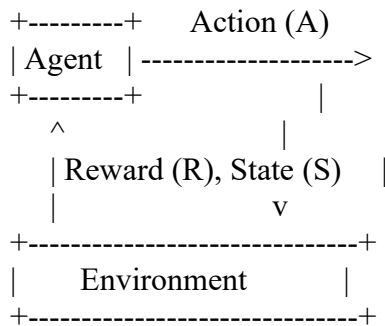
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Reinforcement Learning Framework

Definition

Reinforcement Learning (RL) is a branch of Machine Learning where an **agent** learns by interacting with an **environment**. The agent performs actions, receives rewards or penalties, and gradually learns the best strategy (policy) to maximize the total reward.

Agent–Environment Interaction



Components

- **Agent:** Learner or decision maker.
- **Environment:** The world in which the agent operates.
- **State (S):** Current situation of the environment.
- **Action (A):** Decision taken by the agent.
- **Reward (R):** Feedback after taking an action.
- **Policy (π):** Strategy used by the agent.

Importance of States, Actions and Rewards

- **State** tells the current condition.
- **Action** changes the state.
- **Reward** indicates whether the action was good or bad.

Cumulative Reward

The goal is **not** to maximize immediate reward but the **total future reward**.

Formula:

where

- γ = Discount factor
- G = Total cumulative reward

Example

A robot may choose a longer path because it gives a larger total reward than a short dangerous path.

Simple code

```
state = 0
```

```
for step in range(5):
```

```
    action = 1          # Move right
```

```
    next_state = state + action
```

```
    if next_state == 5:
```

```

        reward = 10
    else:
        reward = -1

    print("State:", state,
          "Action:", action,
          "Reward:", reward)

    state = next_state

```

The screenshot shows a Python IDE with two windows. The left window displays a script for a simple environment. The right window shows the output of the script, which includes a restart message and a list of state-action-reward tuples.

```

state = 0

for step in range(10):
    action = 1  # Move right
    next_state = state + action
    reward = 10
    print("State:", state,
          "Action:", action,
          "Reward:", reward)
    state = next_state

```

```

Python 3.10.9 (tags/v3.10.9:60000a2, Jan 11 2023, 16:13:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "help" above for more information.

===== RESTART: C:/Users/chaht/App
Data/Local/Programs/Python/Python310/m3-at1 ril.py =====
State: 0 Action: 1 Reward: -1
State: 1 Action: 1 Reward: -1
State: 2 Action: 1 Reward: -1
State: 3 Action: 1 Reward: -1
State: 4 Action: 1 Reward: 10

```

2. Markov Decision Process (MDP)

Definition

An MDP is a mathematical framework used to model sequential decision-making problems. It is represented as

Components

1. State Space (S)

All possible situations.

Example:

Robot position in a maze.

2. Action Space (A)

Possible actions.

Example

- Left
- Right
- Up
- Down

3. Transition Probability (P)

Probability of moving to the next state.

Example

Robot moves right successfully with probability 0.9.

4. Reward Function (R)

Reward received after taking an action.

Example

Goal = +10

Obstacle = -5

Normal step = -1

5. Discount Factor (γ)

Determines importance of future rewards.

$0 \leq \gamma \leq 1$

- $\gamma = 0 \rightarrow$ Immediate rewards only
- $\gamma = 1 \rightarrow$ Future rewards equally important

Example

Robot Navigation

State $\rightarrow$ Robot location

Actions $\rightarrow$ Move

Reward $\rightarrow$ Reach goal

Transition $\rightarrow$ Robot movement

Simple code

```
states = ["Start", "Middle", "Goal"]
```

```
actions = ["Right"]
```

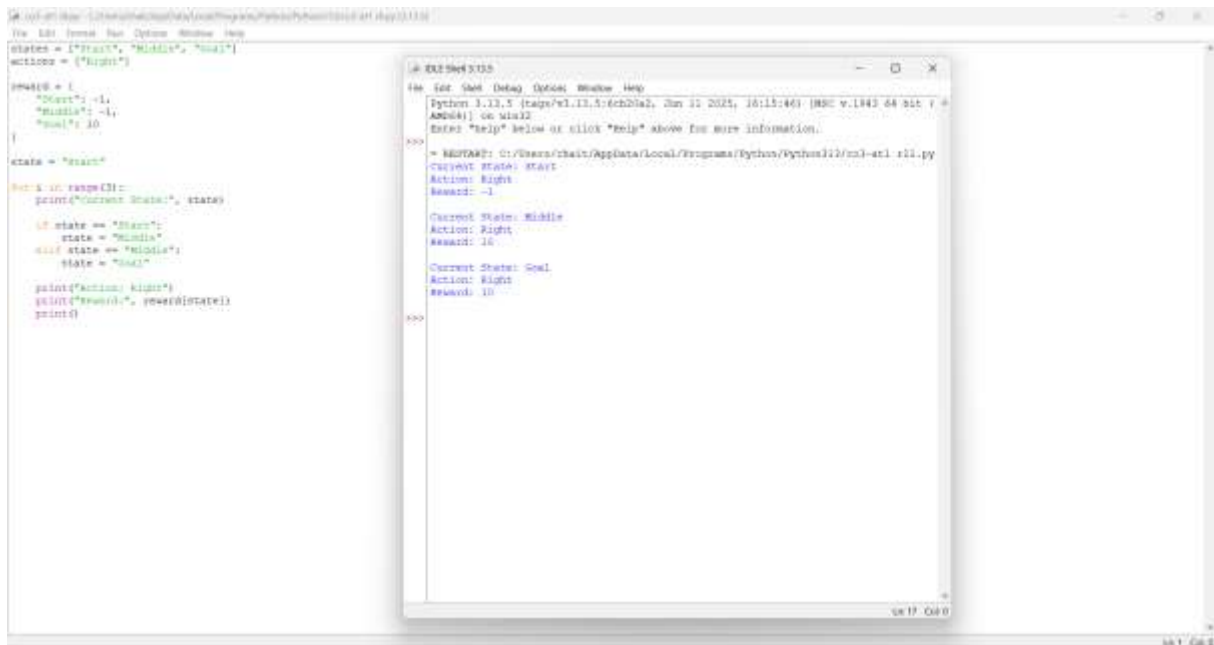
```
reward = {  
    "Start": -1,  
    "Middle": -1,  
    "Goal": 10  
}
```

```
state = "Start"
```

```
for i in range(3):  
    print("Current State:", state)
```

```
    if state == "Start":  
        state = "Middle"  
    elif state == "Middle":  
        state = "Goal"
```

```
    print("Action: Right")  
    print("Reward:", reward[state])  
    print()
```

The image shows two overlapping windows from a Windows operating system. The background window is a text editor with a Python script. The script defines three states: 'Start', 'Middle', and 'Goal', and one action: 'Right'. It includes a main loop that prints the current state and action, and a function to calculate the next state and reward based on the current state and action. The foreground window is a command prompt showing the execution of the script. It displays the current state as 'Start', the action as 'Right', and the reward as -1. It then shows the current state as 'Middle', the action as 'Right', and the reward as 10. Finally, it shows the current state as 'Goal', the action as 'Right', and the reward as 10.

3. Policy, Value Functions and Bellman Equation

Policy

A policy tells the agent what action to perform.

Example

If robot is at position A,

Policy → Move Right.

State Value Function

Expected reward from a state.

Measures

"How good is this state?"

Action Value Function

Expected reward after taking an action.

Measures

"How good is this action?"

Bellman Equation

For State Value

For Q-value

Importance

Bellman Equation updates value estimates repeatedly until convergence.

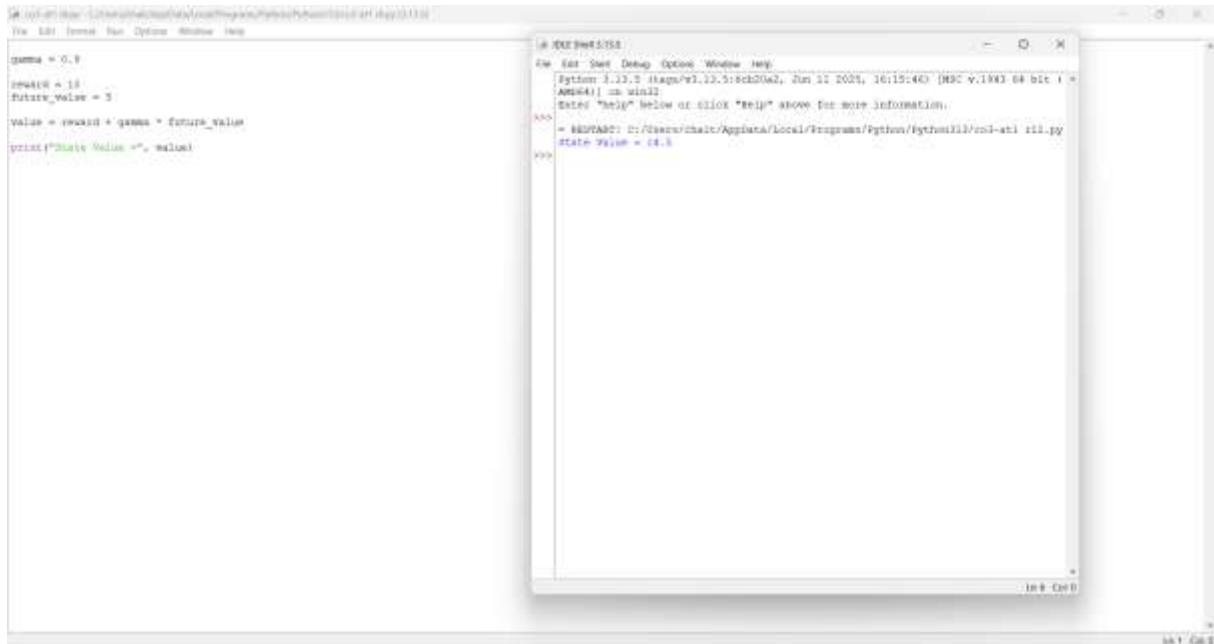
Simple code

gamma = 0.9

reward = 10

future_value = 5

```
print("State Value =", value)
```



4. Exploration vs Exploitation

Exploration

Trying new actions.

Advantages

- Finds better solutions
- Avoids local optimum

Exploitation

Uses the best-known action.

Advantages

- High immediate reward
- Fast decisions

ϵ -Greedy Strategy

- Probability $\varepsilon \rightarrow$ Random action
- Probability $1-\varepsilon \rightarrow$ Best action

Example

 $\varepsilon = 0.1$

10% random

90% best action

Softmax Strategy

Each action gets a probability.

Better actions have higher probability.

Formula

Comparison

ϵ -Greedy

Simple

Random exploration

Easy to implement

Softmax

More balanced

Probability-based

Better performance

Simple code

```
import random
```

```
q_values = [2, 5, 3]
```

```
epsilon = 0.2
```

```
if random.random() < epsilon:
```

```
    action = random.randint(0, 2)
```

```
    print("Exploration")
```

```
else:
```

```
    action = q_values.index(max(q_values))
```

```
    print("Exploitation")
```

```
print("Selected Action:", action)
```



5. Reward Shaping

Definition

Reward shaping adds extra rewards to guide learning faster.

Example

Maze Navigation

Without shaping

Goal = +100

Else = 0
Learning becomes slow.
With shaping
Move closer = +2
Move away = -2
Goal = +100
The agent learns much faster.

Advantages

- ✓ Faster convergence
- ✓ Better exploration
- ✓ Reduces training time
- ✓ Helps avoid poor policies

Preventing Suboptimal Policies

Without shaping
Robot may wander randomly.
With shaping
Robot receives hints and reaches goal efficiently.

Simple code

```
position = 0
goal = 5

while position < goal:

    old_distance = goal - position

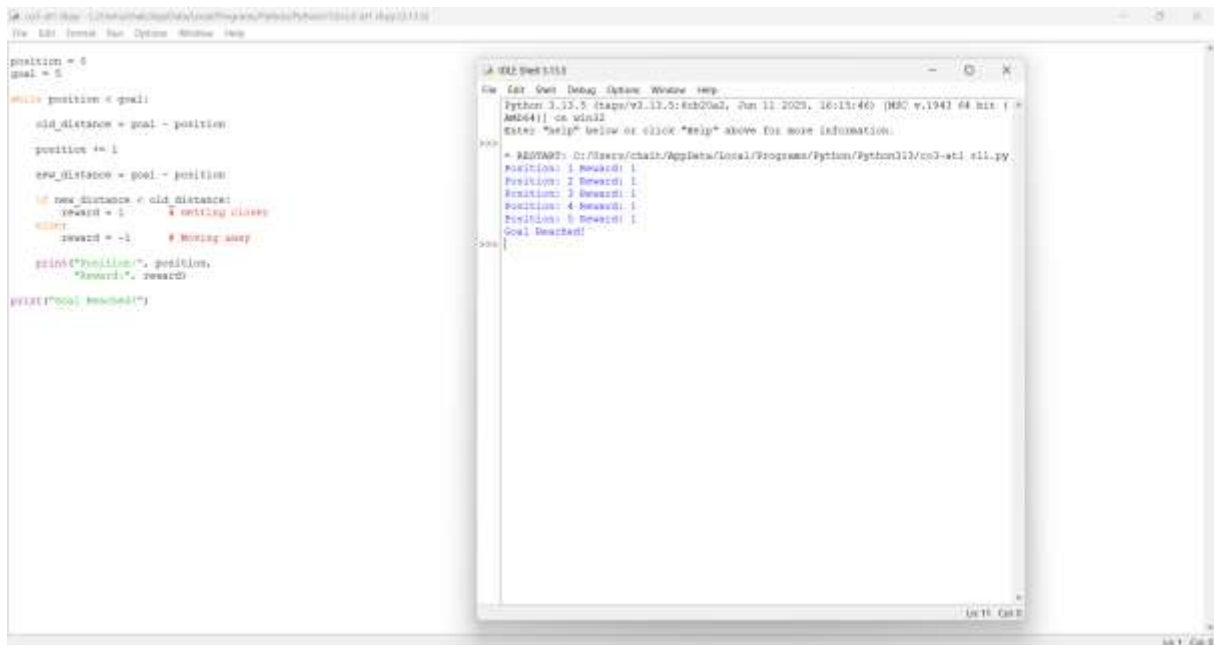
    position += 1

    new_distance = goal - position

    if new_distance < old_distance:
        reward = 1    # Getting closer
    else:
        reward = -1   # Moving away

    print("Position:", position,
          "Reward:", reward)

print("Goal Reached!")
```

The image shows a screenshot of a code editor with a Python script and a terminal window. The script is a simple reinforcement learning simulation where a robot moves towards a goal. The terminal shows the execution of the script, displaying the robot's position and the reward at each step.

```
position = 0
goal = 5

while position < goal:
    old_distance = goal - position
    position += 1
    new_distance = goal - position
    if new_distance < old_distance:
        reward = 1 # getting closer
    else:
        reward = -1 # moving away
    print("Position:", position, "Reward:", reward)

print("Goal Reached!")
```

```
Python 3.11.2 (tags/v3.11.2:844d24e, Jun 11 2023, 16:11:46) [MSC v.1942 64 bit (AMD64)] on win32
Enter "help()" below or click "help" above for more information.

>>>
* RESTART: C:/Users/chaib/AppData/Local/Programs/Python/Python311/python.exe
Position: 1 Reward: 1
Position: 2 Reward: 1
Position: 3 Reward: 1
Position: 4 Reward: 1
Position: 5 Reward: 1
Goal Reached!
>>>
```

6. Challenges of RL in Robotics

1. Safety

Wrong actions may damage robots.

Example

Industrial robot colliding with objects.

2. Sample Efficiency

RL needs thousands of trials.

Real robots cannot perform millions of experiments.

3. Environment Variability

Lighting

Weather

Surface conditions

Sensor noise

All affect learning.

Other Challenges

- Expensive hardware
- Slow learning
- High computational cost

Solutions

Simulation

Transfer Learning

Safe RL

Domain Randomization

Program:

position = 0

obstacle = 3

goal = 5

while position < goal:

 position += 1

 if position == obstacle:

 reward = -10

 print("Obstacle! Reward:", reward)

 position -= 1

 elif position == goal:

 reward = 10

 print("Goal Reached! Reward:", reward)

 else:

 reward = 1

 print("Robot Position:", position,

 "Reward:",

reward)

```
Python 3.10.0 Shell
File Edit Shell Run Options Window Help
position = 0
obstacle = 3
goal = 5

while position < goal:
    position += 1
    if position == obstacle:
        reward = -10
        print("Obstacle! Reward:", reward)
        position -= 1
    elif position == goal:
        reward = 10
        print("Goal Reached! Reward:", reward)
    else:
        reward = 1
        print("Robot Position:", position,
              "Reward:", reward)
```

7. Discount Factor (γ)

Definition

Discount factor determines importance of future rewards.

$$0 \leq \gamma \leq 1$$

Small γ (0.1)

Focuses on immediate rewards.

Example

Robot quickly collects nearby object.

Large γ (0.99)

Focuses on future rewards.

Robot takes longer but safer path.

Role in Episodic Tasks

Episodes end after reaching goal.

Large γ encourages planning.

Small γ encourages short-term decisions.

Example

Chess

Large γ is preferred because future moves matter.

Simple code

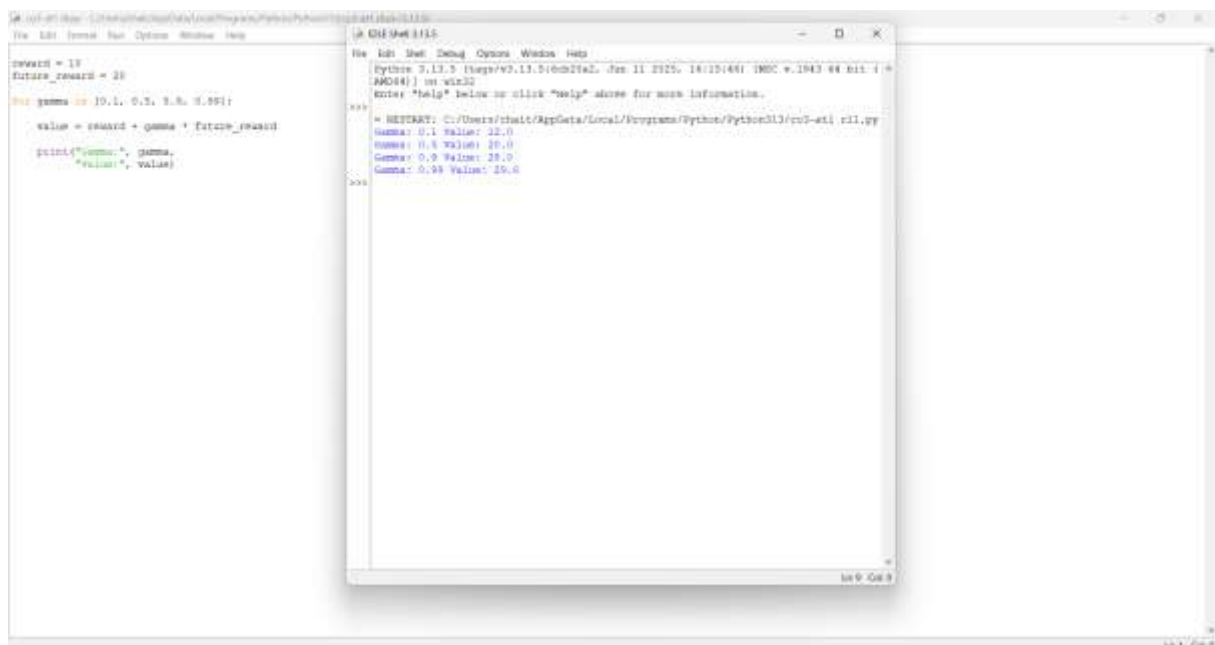
```
reward = 10
```

```
future_reward = 20
```

for gamma in [0.1, 0.5, 0.9, 0.99]:

```
    value = reward + gamma * future_reward
```

```
    print("Gamma:", gamma,  
          "Value:", value)
```



The image shows a screenshot of a Python script being executed in a terminal window. The script defines a reward of 10 and a future_reward of 20. It then iterates over a list of gamma values: [0.1, 0.5, 0.9, 0.99]. For each gamma value, it calculates the value using the formula $value = reward + gamma * future_reward$ and prints the result. The output shows that as gamma increases, the value increases, with gamma 0.99 resulting in the highest value of 20.9.

```
reward = 10
future_reward = 20

for gamma in [0.1, 0.5, 0.9, 0.99]:
    value = reward + gamma * future_reward
    print("Gamma:", gamma,
          "Value:", value)
```

```
Python 3.13.5 Shell [Python 3.13.5 (tags/v3.13.5:6b27ba2, Jan 11 2025, 16:25:46) (MSC v.1943 64 bit)
AMD64] on win32
Enter "help" to see how to use "help" above for more information.

>>>
>>> = RESTART: C:\Users\chait\AppData\Local\Programs\Python\Python313\cmd.exe
Gamma: 0.1 Value: 22.0
Gamma: 0.5 Value: 25.0
Gamma: 0.9 Value: 28.0
Gamma: 0.99 Value: 29.0
>>>
```

8. Model-Free vs Model-Based RL

Model-Free RL

No knowledge of environment.

Learns directly.

Examples

- Q-Learning
- SARSA
- DQN

Advantages

- Easy
- Works in unknown environments

Disadvantages

- Slow
- Needs many samples

Model-Based RL

Builds a model of the environment.

Examples

- Value Iteration
- Policy Iteration

Advantages

- Sample efficient
- Better planning

Disadvantages

- Difficult to build model
- Computationally expensive

Comparison

Model-Free	Model-Based
No model	Uses model
Slow	Fast planning
More data	Less data
Better for unknown worlds	Better for known environments

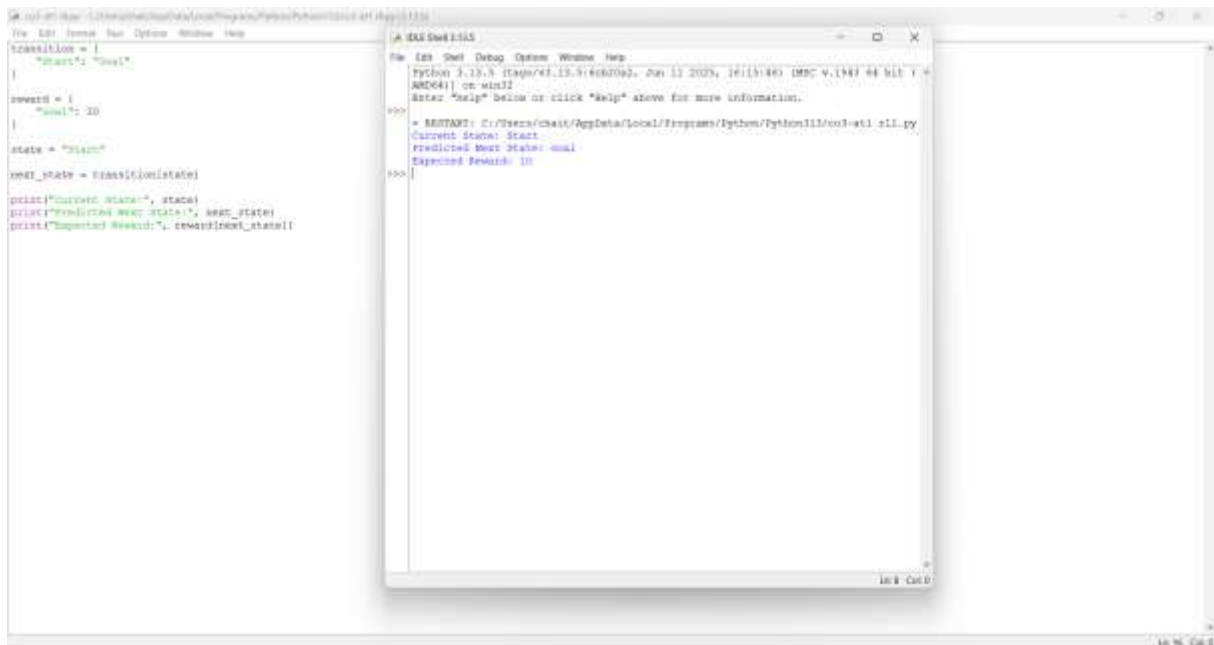
```
transition = {  
    "Start": "Goal"  
}
```

```
reward = {  
    "Goal": 10  
}
```

```
state = "Start"
```

```
next_state = transition[state]
```

```
print("Current State:", state)  
print("Predicted Next State:", next_state)  
print("Expected Reward:", reward[next_state])
```



9. Q-Learning Update Rule

Formula

Where

α = Learning rate

γ = Discount factor

Example

Current Q = 5

Reward = 10

$\gamma = 0.9$

Maximum future Q = 8

$\alpha = 0.5$

Calculation

Target

Difference

$17.2 - 5 = 12.2$

Update

$5 + 0.5 \times 12.2$

$= 11.1$

New Q-value = **11.1**

Simple code

q = 5

reward = 10

future_q = 8

alpha = 0.5

gamma = 0.9

```

new_q = q + alpha * (
    reward + gamma * future_q - q
)

```

```
print("Old Q-value:", q)
print("New Q-value:", new_q)
```

Convergence

Repeated updates gradually converge to the optimal Q-values.

Conditions

- Enough exploration
- Proper learning rate
- Infinite visits to all state-action pairs



10. On-Policy vs Off-Policy Learning

On-Policy

Learns using the same policy that generates actions.

Example

SARSA

Advantages

- Stable
- Safe learning

Disadvantages

- Learns slowly

Off-Policy

Learns using a different policy.

Example

Q-Learning

Advantages

- Faster learning
- Finds optimal policy

Disadvantages

- Can be unstable

Comparison Table

On-Policy	Off-Policy
Learns current policy	Learns optimal policy
Example: SARSA	Example: Q-Learning
Safer	Faster
More stable	Less stable
Less exploration	Better exploration

SARSA – On-Policy

$q = 5$

reward = 10

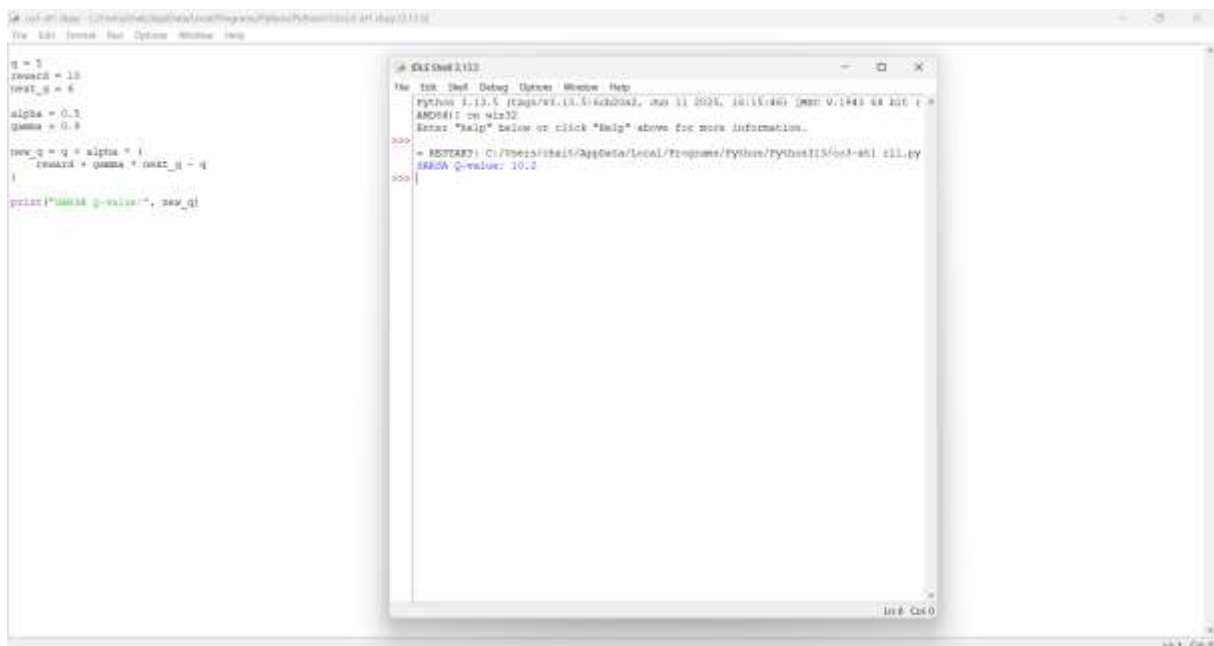
next_q = 6

$\alpha = 0.5$

$\gamma = 0.9$

```
new_q = q + alpha * (  
    reward + gamma * next_q - q  
)
```

```
print("SARSA Q-value:", new_q)
```



The image shows a screenshot of a Python IDE with two windows. The background window is a script editor containing the following code:

```
q = 5  
reward = 10  
next_q = 6  
  
alpha = 0.5  
gamma = 0.9  
  
new_q = q + alpha * (  
    reward + gamma * next_q - q  
)  
  
print("SARSA Q-value:", new_q)
```

The foreground window is a terminal or command prompt showing the execution of the script. It displays the following output:

```
Python 3.13.5 (tags/v3.13.5:4303a2, Jun 11 2023, 18:15:46) [AMD64] on win32  
Enter "help" below or click "Help" above for more information.  
  
=> RESTART: C:\Users\ibrah\AppData\Local\Programs\Python\Python113\python.exe  
SARSA Q-value: 10.2
```